



南京理工大学

6 回溯法

Backtracking



引例1

- 猜谜

头上红冠不用裁，满身雪白走将来。

平生不敢轻言语，一叫千门万户开。

--打一动物



南京理工大学



画鸡 [明] 唐寅

头上红冠不用裁，满身雪白走将来。

平生不敢轻言语，一叫千门万户开。



图虫创意 stock.tuchong.com



引例2

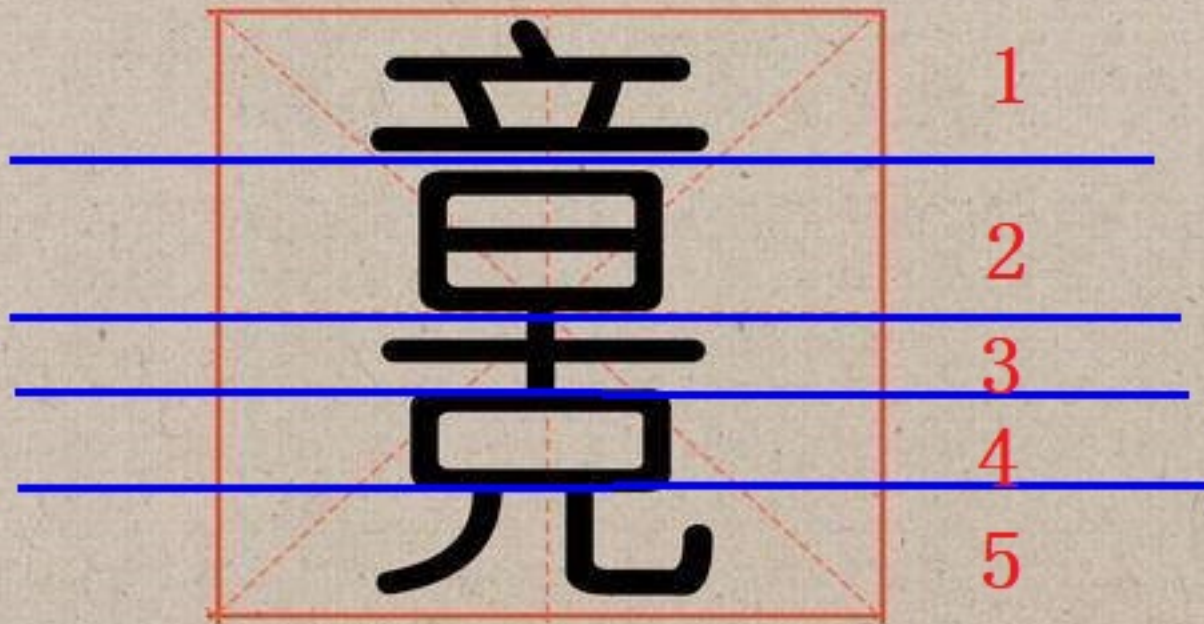


这张图，你能找出几个字？

一名京人原创



引例2

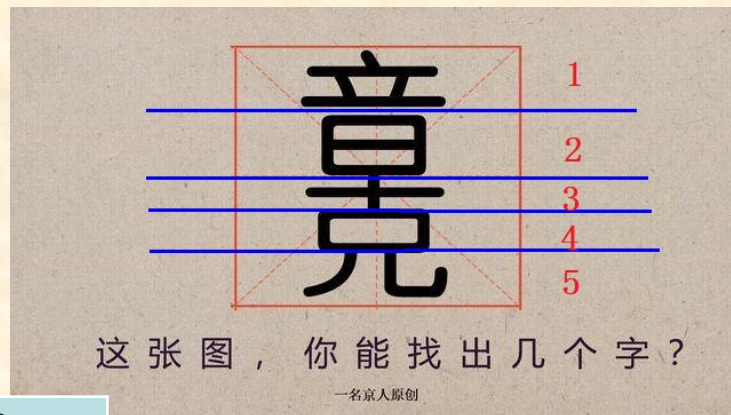


这张图，你能找出几个字？

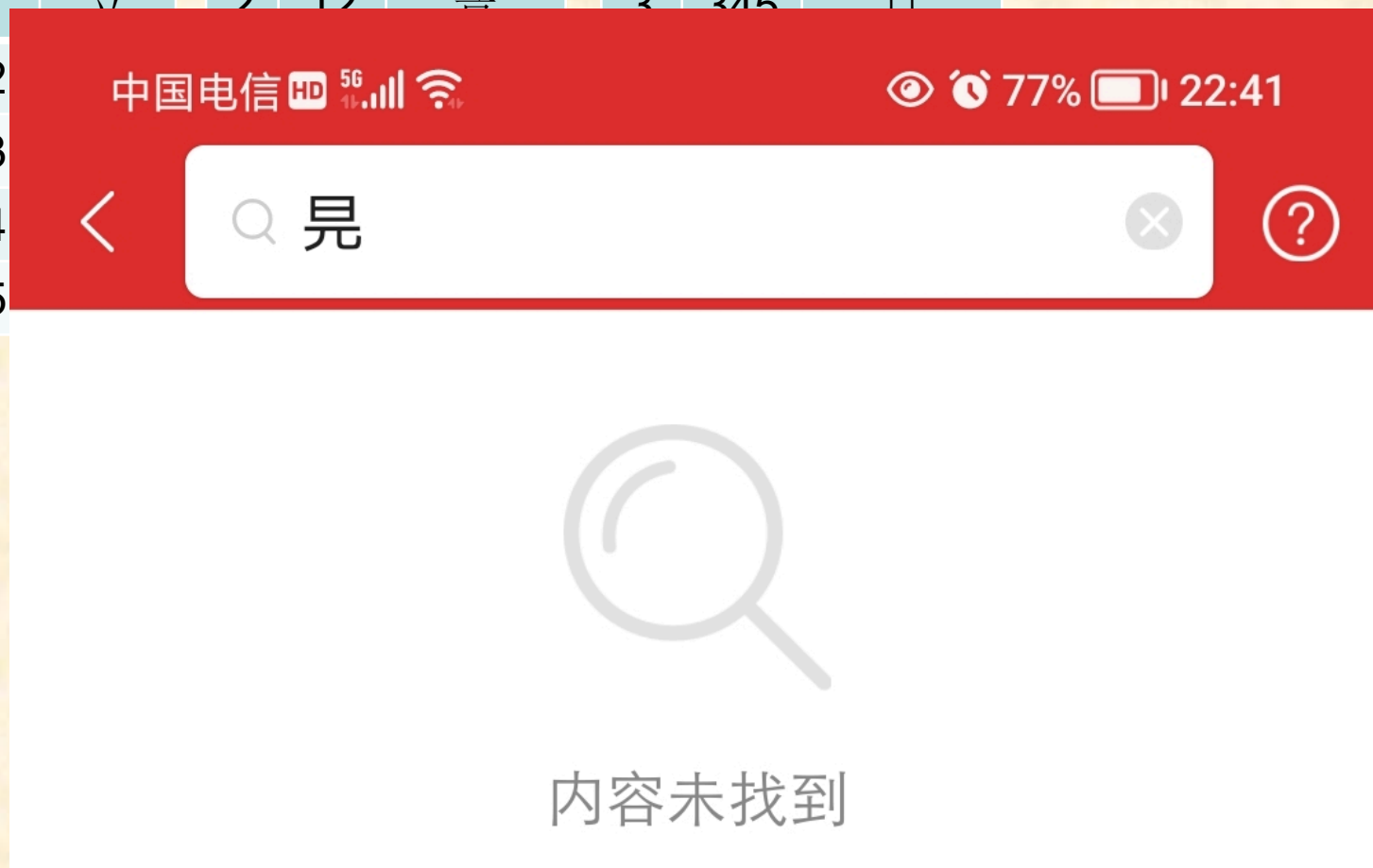
一名京人原创



引例2



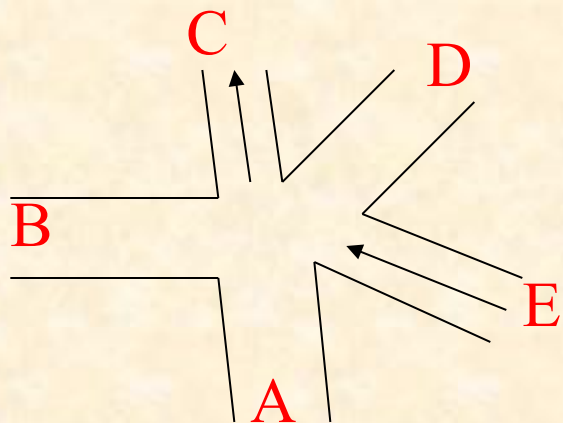
1	1	立	2	12	音	3	345	古
	2							
	3							
	4							
	5							



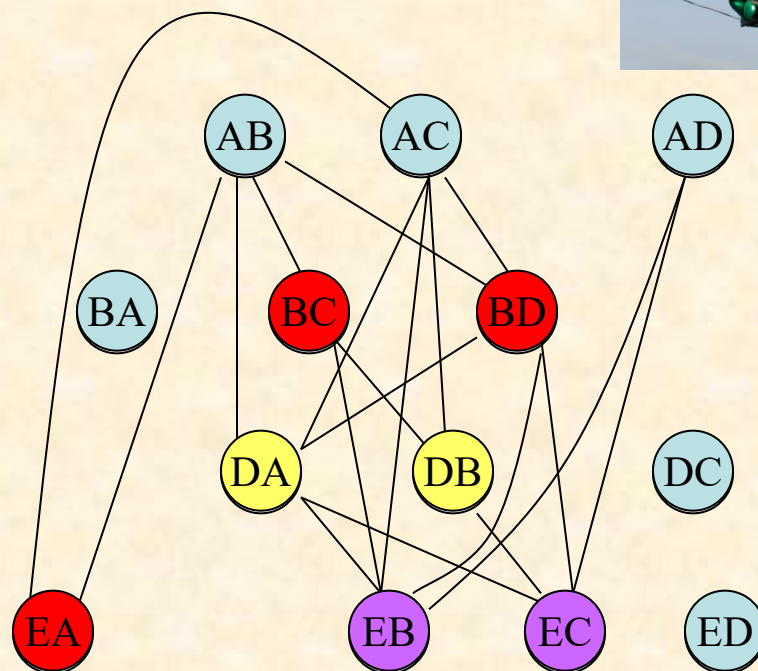
X
X
X
X
X
X



引例3：多叉路口交通灯管理系统



13条通路，考察任意两条通路是否互相碰撞，在78种情况下有20种情况会碰撞（用连线表示）

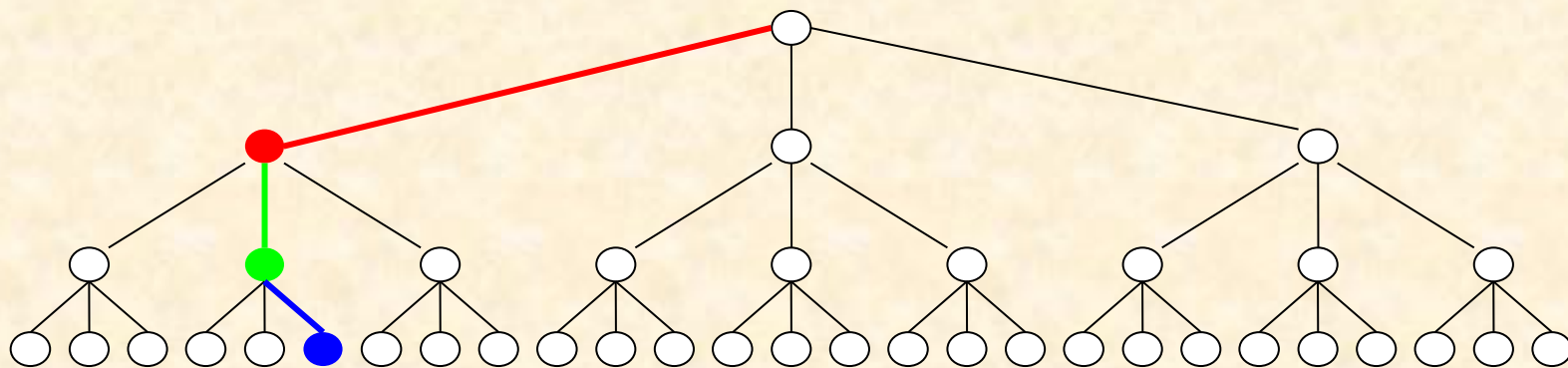


设置交通灯的问题等价于对图的顶点着色问题：要求对图上的每一个顶点染一种颜色，并且要求有线相连的两个顶点不能具有相同颜色，而总的颜色种类应尽可能地少。



引例

- 图的3着色问题
- 给定一个无向图 $G=(V,E)$ ，及3种颜色 $\{1,2,3\}$ ，现要为图的每个顶点着色。每顶点只能着一种颜色，并且要求相邻的顶点具有不同的颜色。
- 一个具有 n 个顶点的图，可用一个 n 维的向量 $(c_1, c_2, \dots, c_n)$ 表示一种着色方案， $c_i \in \{1, 2, 3\}$, $i=1, 2, \dots, n$ ，共有 3^n 种可能的着色，可用一棵完全的3叉树表示。
- 下图为有3个顶点的所有可能的着色搜索树，从根到叶结点的每条路径表示一种着色方案。



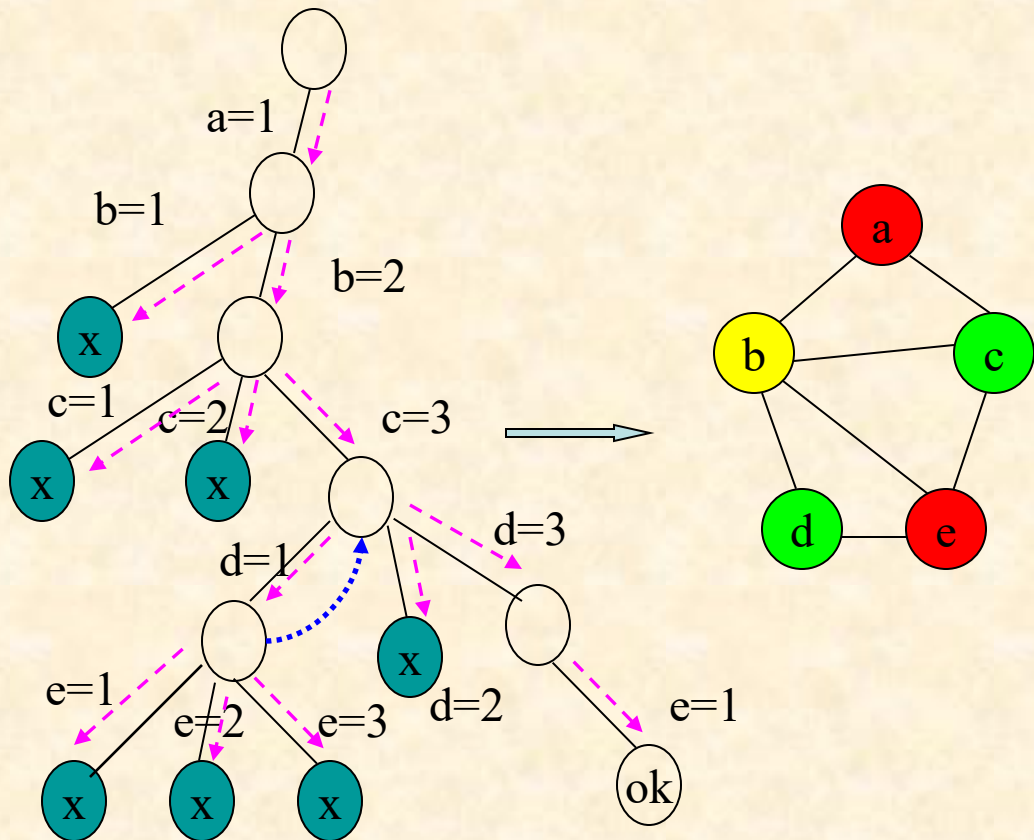
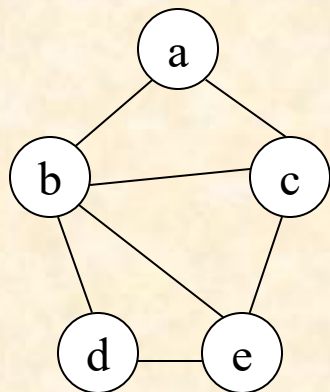
几个概念

- 问题的解向量：问题的解能够表示成一个 n 维向量 $(x_1, x_2, \dots, x_n)$ 的形式。
- 显式约束：对分量 x_i 的取值限定。
- 隐式约束：为满足问题的解而对不同分量之间施加的限定。
- 问题的解空间：对于问题的一个实例，解向量满足**显式约束**条件的所有 n 维向量，构成了该问题实例的一个解空间。



回溯法的工作原理

- 回溯法的基本做法是搜索，是一种组织得井井有条、能避免不必要搜索的穷举式搜索法。
- 回溯法按**深度优先策略**搜索问题的解空间树。算法搜索至解空间树的任意一结点时，先判断该结点是否可能包含问题的解：1) 如果肯定不包含，则跳过这个结点；2) 如果可能包含，进入该子树，继续按深度优先策略搜索；3) 若某结点 i 的所有子结点都不可能包含问题的解，则回溯到 i 的父结点，生成下一个结点，继续搜索。



“合法着色” — 全部顶点都已经被着色，且没有两个相邻的顶点是同样的颜色。
“部分着色” — 一部分顶点还未着色；在已经着色的顶点中，没有两个相邻的顶点是同样的颜色。



递归回溯

输入：无向图 $G=(V,E)$

输出：G的顶点的3着色 $c[1\dots n]$ ，其中每个 $c[j]$ 为1,2或3.

1. for $k \leftarrow 1$ to n
2. $c[k] \leftarrow 0$ //no color
3. end for
4. flag $\leftarrow$ false
5. *graphcolor*(1)
6. if flag then output c
7. else output “no solution”

graphcolor(k)

1. for color=1 to 3
2. $c[k] \leftarrow$ color
3. if c为合法着色 then flag $\leftarrow$ true and exit
4. else if c是部分着色 then *graphcolor*(k+1)
5. end for



迭代回溯

输入：无向图 $G=(V,E)$

输出：G的顶点的3着色 $c[1...n]$ ，其中每个 $c[j]$ 为1,2或3.

1. for $k \leftarrow 1$ to n
2. $c[k] \leftarrow 0$
3. end for
4. flag $\leftarrow$ false
5. $k \leftarrow 1$ //start from v_1
6. while $k \geq 1$
7. while $c[k] \leq 2$ //为第 k 个顶点着色
8. $c[k] \leftarrow c[k] + 1$
9. if c 为合法着色 then flag $\leftarrow$ true and exit
10. else if c 为部分着色 then $k \leftarrow k + 1$ //准备为下一顶点着色, $\rightarrow 7$
11. end while //假设着色既不合法也非部分的，即死结点，试其他色
12. $c[k] \leftarrow 0$ // v_k 试验了所有颜色均失败，当前顶点颜色只好归0，回溯
13. $k \leftarrow k - 1$ //回溯到上一个顶点 (配合第6句的 $k \geq 1$ 来理解)
14. end while //注意：回溯到上一个顶点后， $c[k] + 1$ ，即尝试下一种颜色



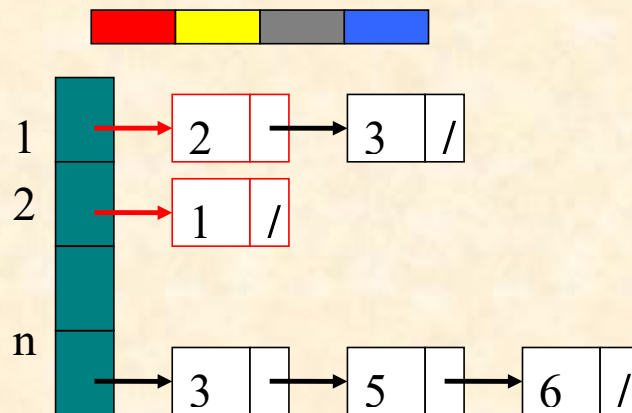
复杂度分析

判断着色 $c[1...n]$ 是否合法的时间耗费:

每个顶点 v_i 都要扫描它的邻边,

看 v_i 与邻接顶点的颜色是否冲突,

每条边扫描 2 次, 共耗时 $\Theta(2m)$



判断着色 $c[1...k]$ ($k < n$) 是否是部分的的时间耗费

由以上分析可知, 每次执行该检查, 耗时 $O(2m)$

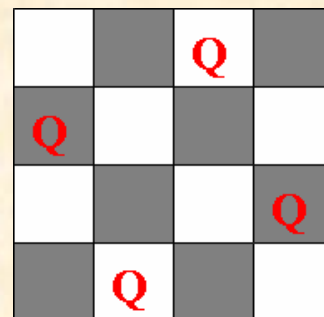
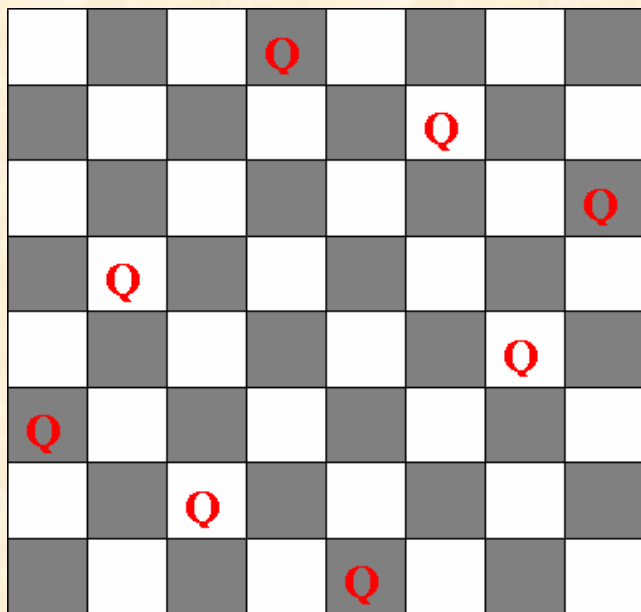
上述两种算法 (递归、迭代) 最坏情况下生成了 $O(3^n)$ 个结点。

因此, 最坏情况下时间耗费 $O(m3^n)$



高斯8皇后问题

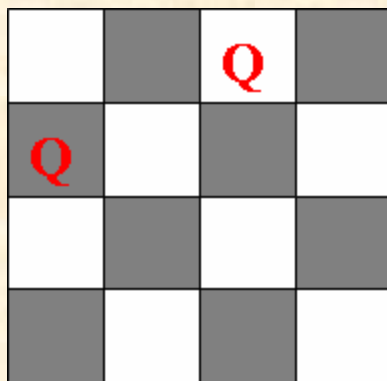
- 著名的数学家高斯在1850年提出：在 8×8 格的国际象棋上摆放八个皇后，使其不能互相攻击，即：任意两个皇后都不能处于同一行、同一列或同一斜线上，问如何摆放？





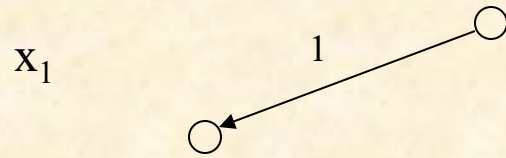
考察 n 皇后问题 $n=4$ 的情形：

- 合法布局：一个不互相攻击的4 皇后布局
- 部分布局：一个不互相攻击的少于4个皇后的布局
- 下图表示一个部分布局，用向量 $(3,1,0,0)$ 表示；





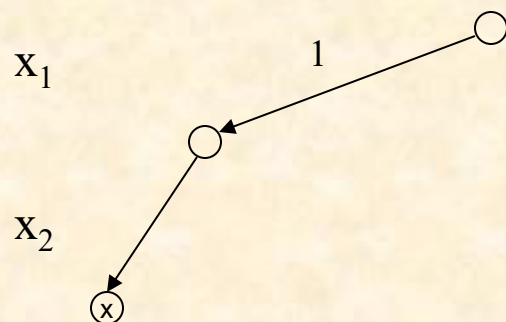
南京理工大学



Q			



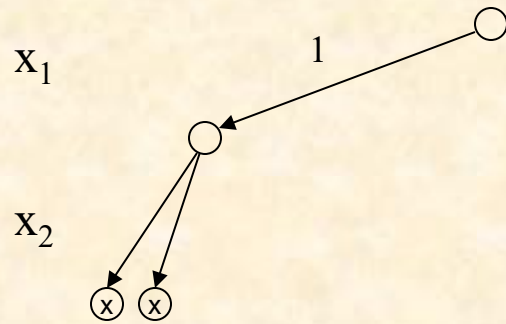
南京理工大学



Q			
X			



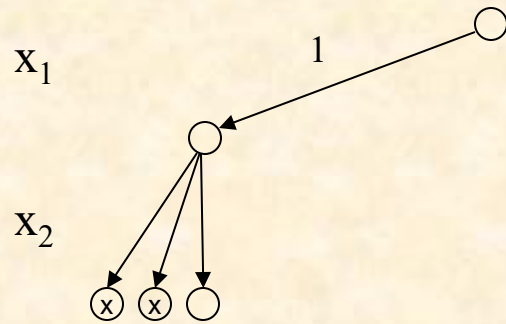
南京理工大学



Q			
X	X		



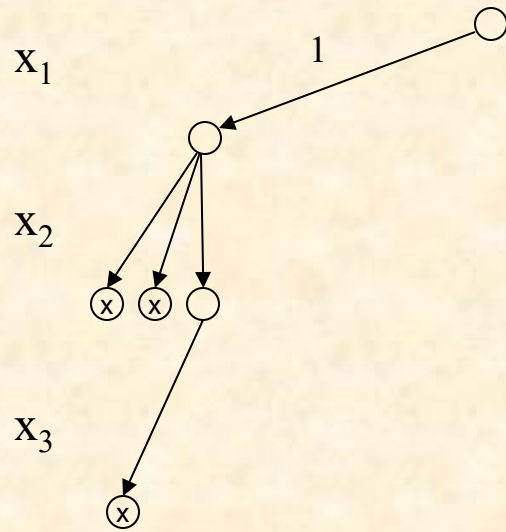
南京理工大学



Q			
X	X	Q	



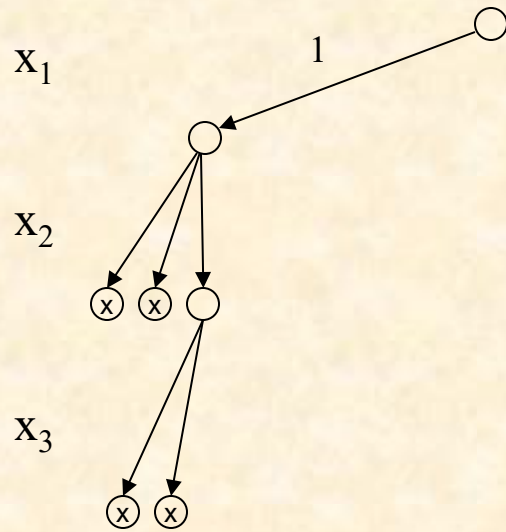
南京理工大学



Q			
X	X	Q	
X			



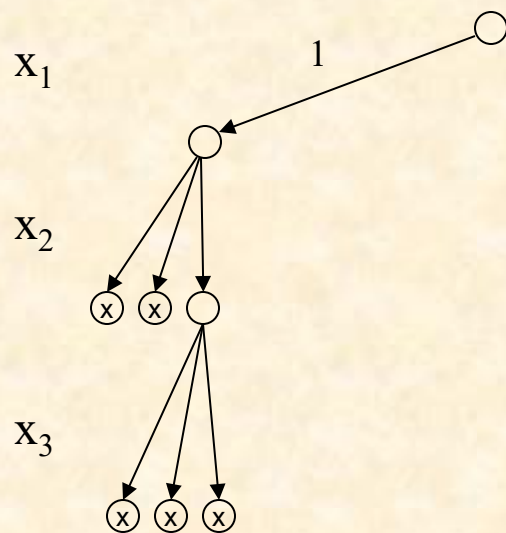
南京理工大学



Q			
X	X	Q	
X	X		



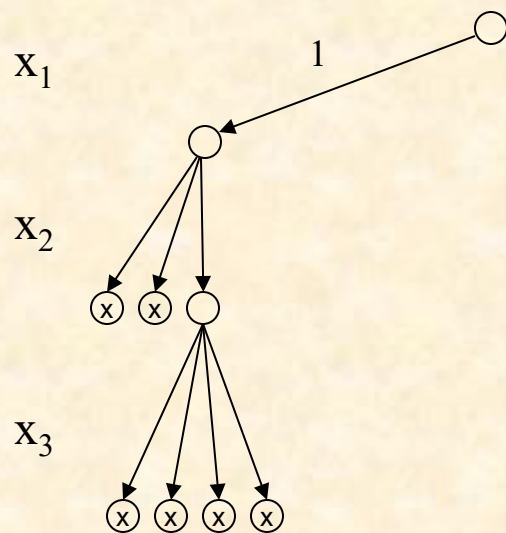
南京理工大学



Q			
X	X	Q	
X	X	X	



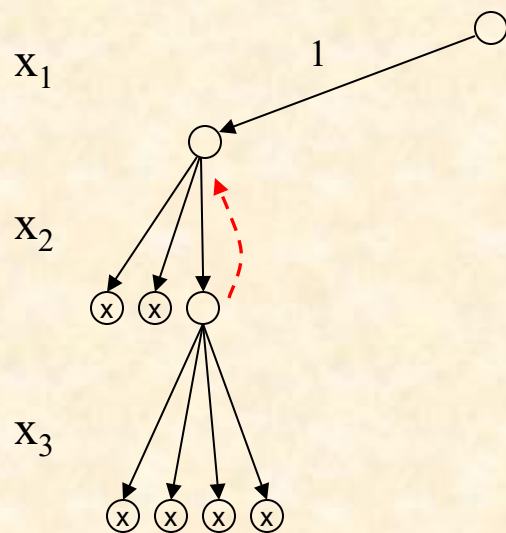
南京理工大学



Q			
X	X	Q	
X	X	X	X



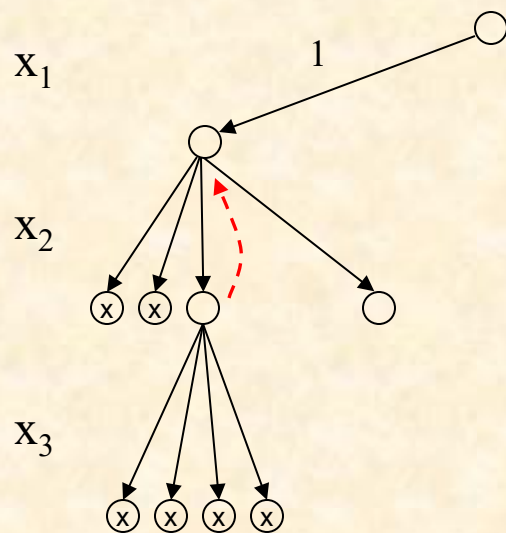
南京理工大学



Q			
X	X	Q	



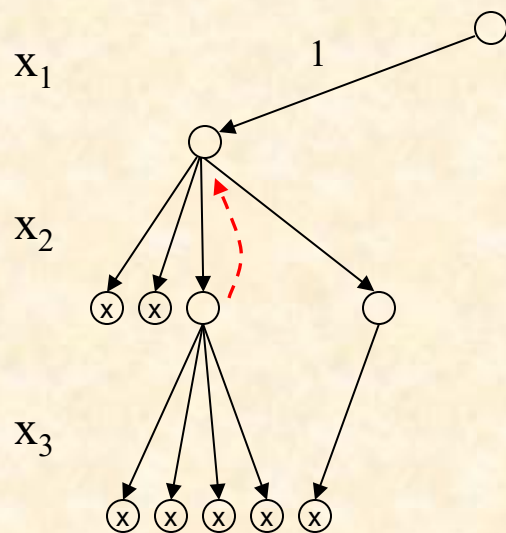
南京理工大学



Q			
X	X	X	Q



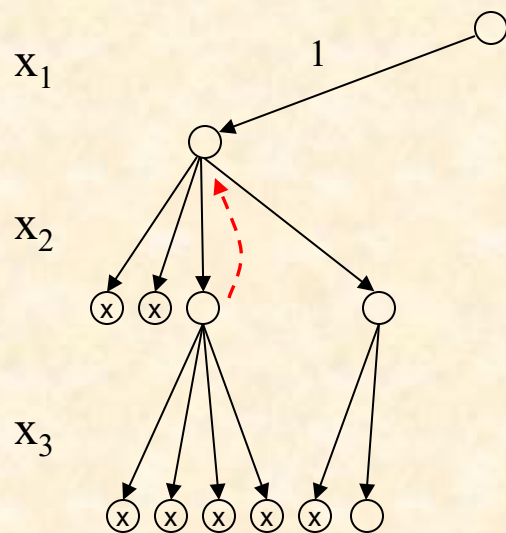
南京理工大学



Q			
			Q
X			



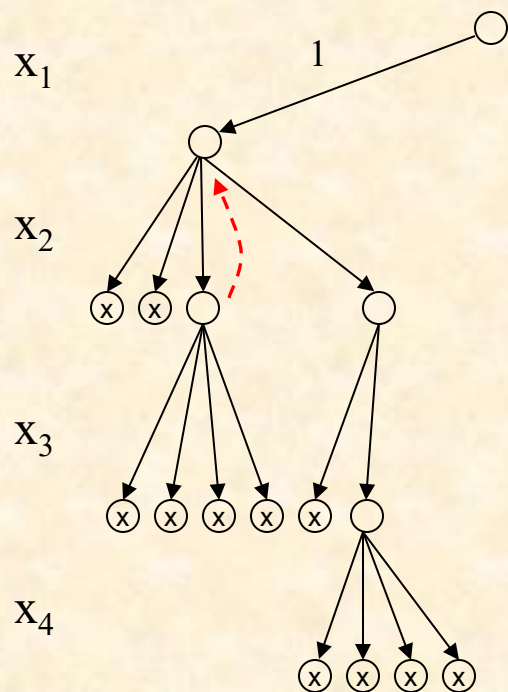
南京理工大学



Q			
			Q
	Q		



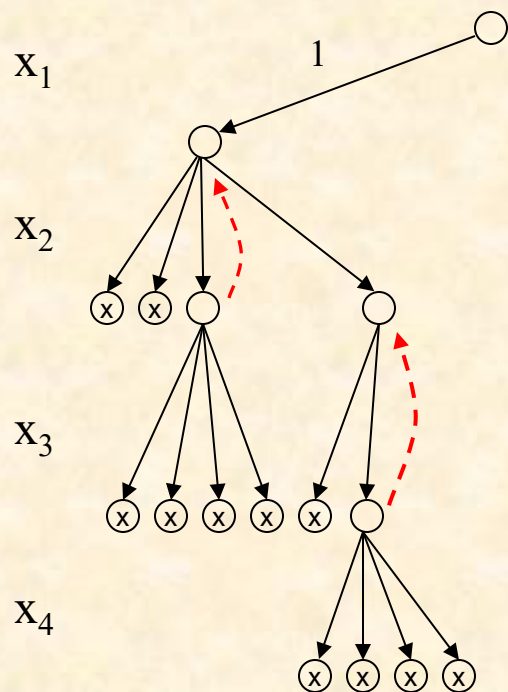
南京理工大学



Q			
			Q
	Q		
X	X	X	X



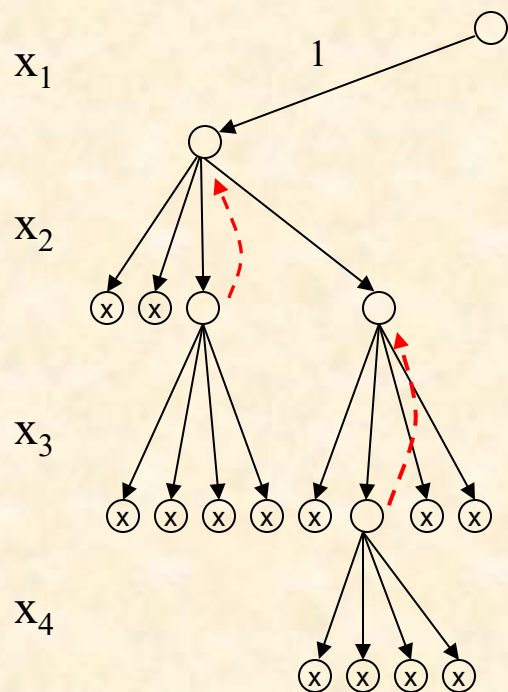
南京理工大学



Q			
			Q
	Q		



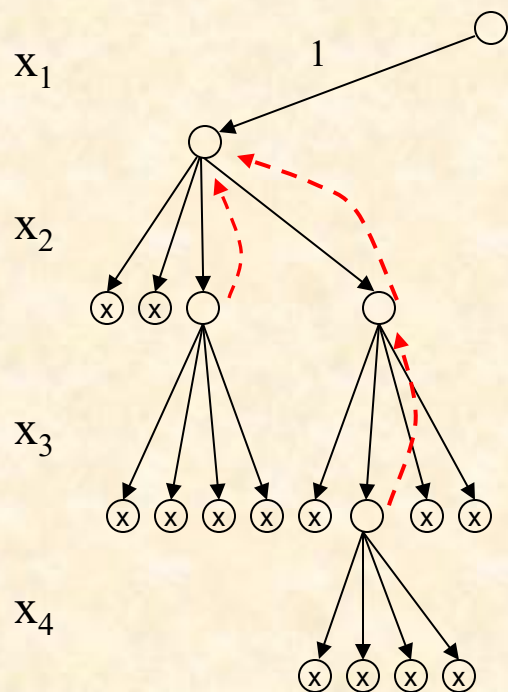
南京理工大学



Q			
			Q
		X	X



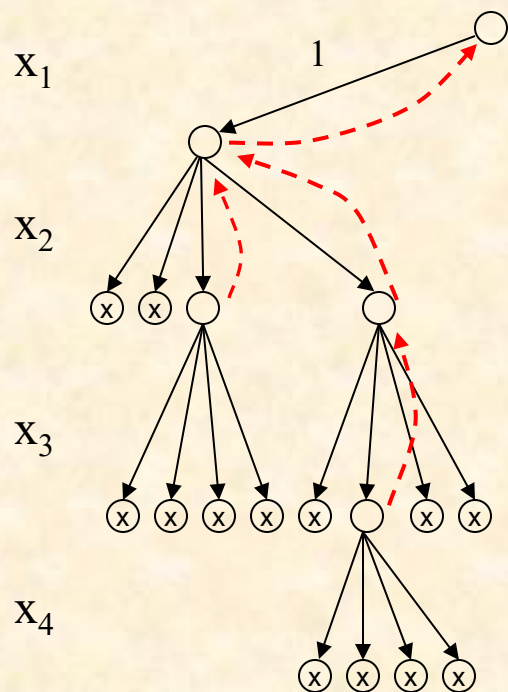
南京理工大学



Q			
			Q



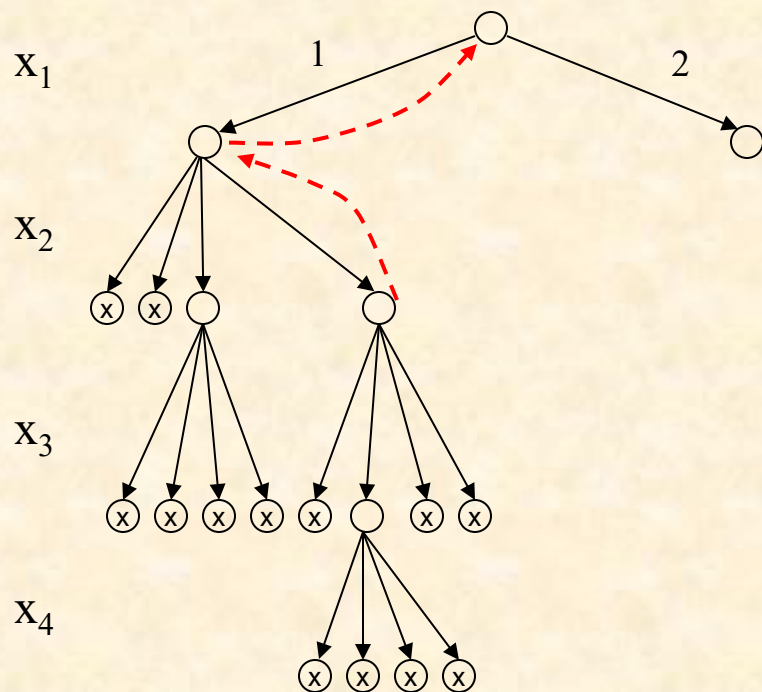
南京理工大学



Q			



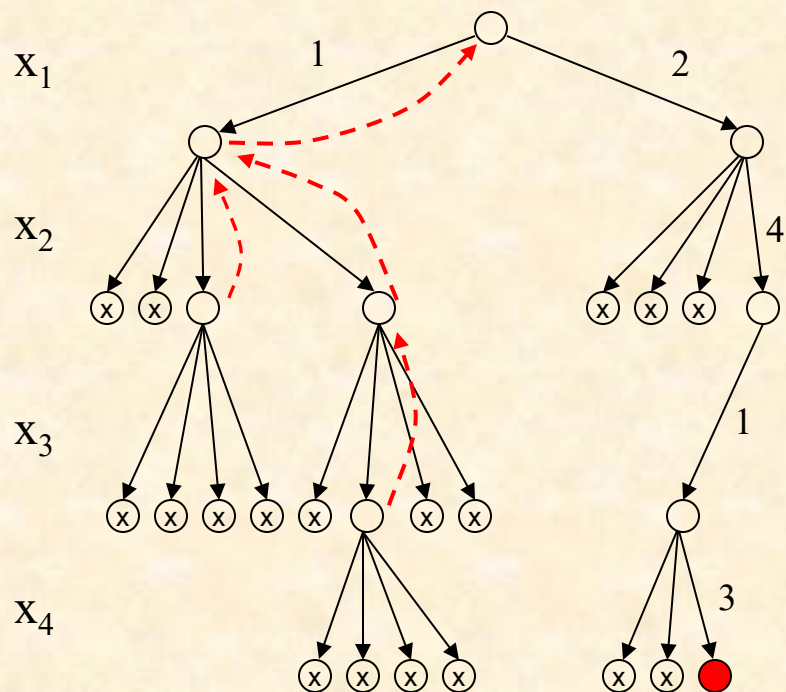
南京理工大学



	Q		



南京理工大学



	Q		
			Q
Q			
		Q	



递归回溯

输入：空

输出：4皇后问题的解向量 $c[1...4]$

1. for $k \leftarrow 1$ to 4
2. $c[k] \leftarrow 0$
3. end for
4. $flag \leftarrow \text{false}$
5. ***SetQueen***(1)
6. if $flag$ then output c
7. else output “no solution”

SetQueen(k)

1. for $x = 1$ to 4
2. $c[k] \leftarrow x$
3. if c 为合法布局 then $flag \leftarrow \text{true}$ and exit
4. else if c 是部分布局 then ***SetQueen***($k+1$)
5. end for



迭代回溯

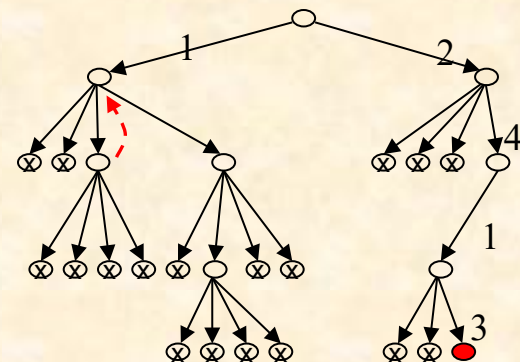
输入：空

输出：4皇后问题的解向量 $c[1...4]$

```
1. for  $k \leftarrow 1$  to  $n$ 
2.    $c[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $k \leftarrow 1$  //从第一个皇后开始
6. while  $k \geq 1$ 
7.   while  $c[k] \leq 3$  //安排第 $k$ 个皇后的位置
8.      $c[k] \leftarrow c[k] + 1$ 
9.     if  $c$ 为合法布局 then  $flag \leftarrow true$  and exit
10.    else if  $c$ 为部分布局 then  $k \leftarrow k + 1$  //准备安排下一个皇后的位置
11.  end while //若布局既不是合法布局，也不是部分布局，则试其他位置
12.  $c[k] \leftarrow 0$  //  $c[k]$ 试验了所有位置均失败,  $c[k]$ 只好归0，回溯
13.  $k \leftarrow k - 1$  //回溯到上一个顶点 (配合第6句的 $k \geq 1$ 来理解)
14. end while
```



算法分析

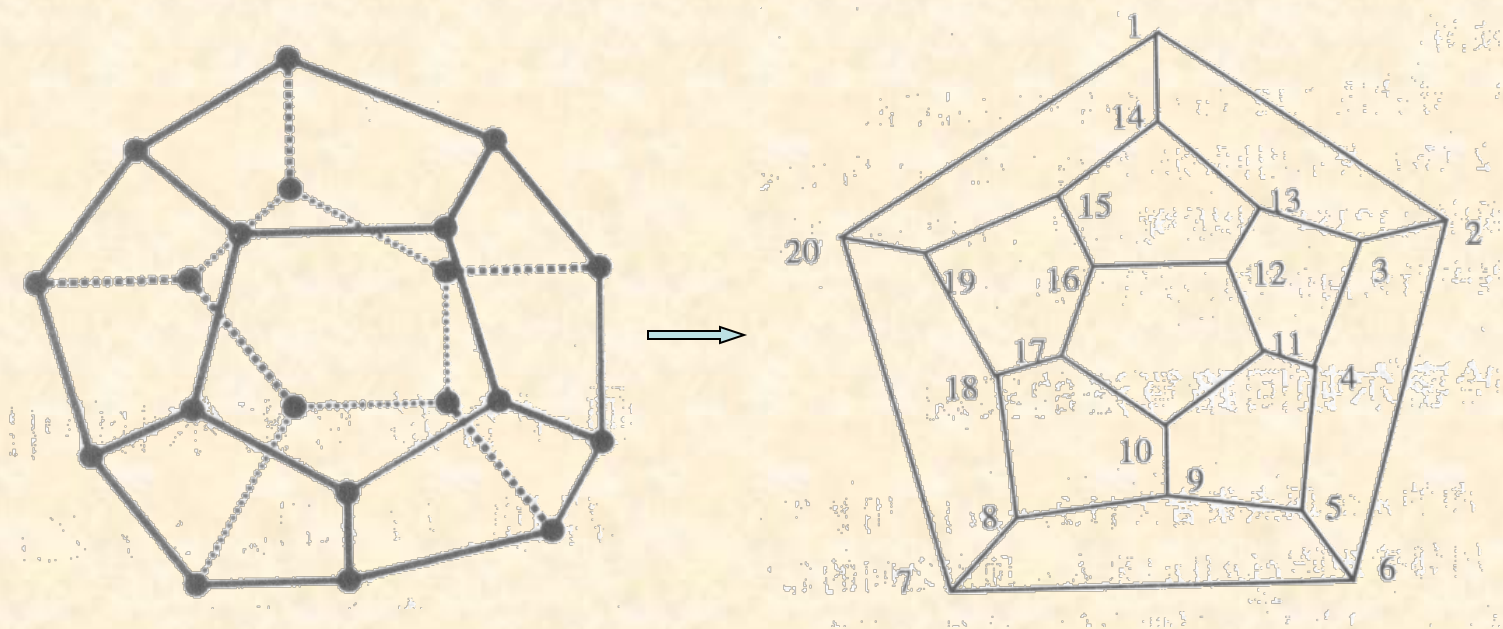


- 算法效率：蛮力方法候选解的个数是 $n!$ ；而使用回溯法，候选解的个数是 n^n ，回溯的效率是否太低了？
- 然而，仔细分析可以发现，回溯法可以极大减少测试次数：
 - 例如，假设前两个皇后冲突，蛮力方法仍旧要测试 $(n-2)!$ 个候选解；而回溯法只要进行一次测试，就可以避免剩余无意义的测试。
 - 所以，尽管在最坏情况下要用 $O(n^n)$ 时间来求解，然后大量实际经验发现，它在有效性上远远超过蛮力方法 $O(n!)$ 。例如在4 皇后问题中，只搜索了256个节点中的19个就找到了解。



哈密顿回路问题

- 1859年，爱尔兰数学家哈密顿提出了著名的周游世界问题。他用正12面体的20个顶点代表20个城市，要求从一个城市出发，经过每个城市恰好一次，然后回到出发城市。





分析

- 假定图 $G=(V,E)$ 的顶点集 $V=\{1,2,\dots,n\}$ ，则哈密顿回路问题的可能解表示为 n 维的向量 $x=(x_1,\dots,x_n)$ ，其中 $x_i \in \{1,2,\dots,n\}$ 。根据题意，有如下约束条件：
 - $(x_i, x_{i+1}) \in E \quad (1 \leq i \leq n-1)$
 - $(x_n, x_1) \in E$
 - $x_i \neq x_j \quad (1 \leq i, j \leq n, i \neq j)$
- 在所有的可能解中，考虑到约束条件 $x_i \neq x_j \quad (1 \leq i, j \leq n, i \neq j)$ ，则可能的解应该是 $(1,2,\dots,n)$ 的一个排列，对应的解空间中有 $n!$ 个叶子结点。



Hamilton回路算法

输入：图 $G=(V,E)$ ，其中 $V=\{1,2,\dots,n\}$

输出：Hamilton回路序列 $x=[x_1,\dots,x_n]$ ，其中 $x_i \in \{1,2,\dots,n\}$

```
1.  $x[1\dots n] \leftarrow 0$ ;  $\text{visited}[1\dots n] \leftarrow \text{false}$ 
2.  $k \leftarrow 1$ ;  $\text{visited}[1] \leftarrow \text{true}$ ;  $x[1] \leftarrow 1$ ; //从顶点1出发
3. while  $k \geq 1$ 
4.    $x[k] \leftarrow x[k] + 1$ 
5.   while  $x[k] \leq n$ 
6.     if  $\text{visited}[x[k]] = \text{false}$  and  $(x[k-1], x[k]) \in E$  break;
7.     else  $x[k] \leftarrow x[k] + 1$  //搜索下一个顶点
8.     if  $x[k] \leq n$  and  $k = n$  and  $(x[k], x[1]) \in E$  then //结束搜索
9.       return  $x[1\dots n]$ ;
10.    else if  $x[k] \leq n$  and  $k < n$  //部分解
11.       $\text{visited}[x[k]] \leftarrow \text{true}$ ;  $k \leftarrow k + 1$ 
12.    else //回溯
13.       $x[k] \leftarrow 0$ ;  $\text{visited}[x[k]] \leftarrow \text{false}$ ;  $k \leftarrow k - 1$ 
14.  end while
15. end while
```




通用回溯方法框架

- 本小节描述通用回溯方法的一般框架，可以作为系统搜索的基本框架，在解决实际问题时，修改该基本框架中的相应部分使之适合实际问题即可。
- 对于以下这样一类问题，可以使用该方法：这类问题的解满足事先定义好的某种约束向量 $(x_1, \dots, x_n)$ ，这里 n 是取决于问题实例的常量。



在回溯法中，每个 x_i 均是属于某个有限集合的，不妨称之为 X_i ，那么回溯法实质上是按照词典顺序考虑笛卡儿积：

$$X_1 \times X_2 \times \dots \times X_n$$

中的元素。

- 算法最初从空向量开始，先选择 X_1 中的最小元素作为 x_1 ；
- 如果 (x_1) 是部分解，那么选择 X_2 中的最小元素作为 x_2 ；
- 如果 (x_1, x_2) 是部分解，则继续考虑 X_3 中的最小元素作为 x_3 ，
- 否则，考虑 X_2 中的第二个元素作为 x_2 。依此类推。



一般说来，当算法已经检测到部分解 $(x_1, x_2, \dots, x_j)$ ，需要继续考虑 $v = (x_1, x_2, \dots, x_j, x_{j+1})$ 时，有以下几种情形：

1. 若 v 表示问题的最终解，算法记录下它作为一个解。如果只需要一个解，算法结束；否则，继续找其它解。
2. 如果 $v = (x_1, x_2, \dots, x_j, x_{j+1})$ 是一个部分解，那么选择集合 X_{j+2} 中未使用过的最小元素作为 x_{j+2} 。(向前搜索)
3. 如果 v 既非最终解，也非部分解，那么会有以下两种情况：
 - a. 如果集合 X_{j+1} 中还有其它未曾使用过的元素，则选择下一个未曾使用过的元素作为 x_{j+1} 。
 - b. 如果集合 X_{j+1} 中没有其它未曾使用过的元素，则回溯，将 X_j 中未曾使用的下一元素作为 x_j ，并将 x_{j+1} 进行重置。如果集合 X_j 中没有未曾使用的元素，那么继续回溯(同样注意要进行重置)。



递归回溯

输入：集合 $X_1, X_2, \dots, X_n$ 清楚或是隐含的描述

输出：解向量 $v = (x_1, x_2, \dots, x_n)$, $0 \leq i \leq n$

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $\text{advance}(1)$
4. if flag then output v
5. else output “no solution”

过程 $\text{advance}(k)$

1. for 每个 $x \in X_k$
2. $x_k \leftarrow x$; 将 x_k 加入 v
3. if c 为最终解 then set $\text{flag} \leftarrow \text{true}$ and exit
4. else if v 是部分解 then $\text{advance}(k+1)$
5. end for



迭代回溯

输入：集合 $X_1, X_2, \dots, X_n$ 清楚或是隐含的描述

输出：解向量 $v = (x_1, x_2, \dots, x_n)$, $0 \leq i \leq n$

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $k \leftarrow 1$
4. while $k \geq 1$
5. while X_k 没有被穷举
6. $x_k \leftarrow X_k$ 里的下一个元素；将 x_k 加入 v
7. if v 为最终解 then set $\text{flag} \leftarrow \text{true}$ and exit
8. else if v 是部分解 then $k \leftarrow k+1$ //前进
9. end while
10. 重置 X_k //一般将 $x_k \leftarrow 0$ ，表示将其重置
11. $k \leftarrow k-1$ //回溯
12. end while
13. if flag then output v
14. else output “no solution”



分枝限界法

- 分枝限界法像回溯法一样生成问题的搜索树，但是，回溯法寻找的是满足给定性质的一个解或解的集合（如3着色问题、8皇后问题），而分枝限界法关心的是目标函数的最大化或最小化。如背包问题，若用回溯法求解，则在解空间搜索树中找到许多可行解，然后从可行解中找到使得背包内物品价值最大化的解，即最优解。然而，利用分枝限界法，无需这样即可找到问题的最优解。

特点：

- 1) 搜索解空间树，按广度优先搜索次序生成结点
- 2) 没有回溯过程
- 3) 适合于求解最优化问题



背包问题

考虑 $n=4$ 的背包问题（背包承重量为 $W=10$ ）：

物品	重量	价值	价值/重量
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

物品已经按照“价值-重量比”降序排列，这样会带来处理上的方便。

估计上界 ub 的一个简单方法是：把已经选择的物品总价值 v ，加上背包剩余承重量 $W-w$ 与剩下可选择的物品的最高“价值-重量比”的乘积，即

$$ub = v + (W-w) \times (v_i/w_i)_{\max}$$

比如，若已装物品 1，则价值上界为 $40+(10-4) \times 6=76$



物品	重量	价值	v/w
----	----	----	-----

1	4	40	10
---	---	----	----

2	7	42	6
---	---	----	---

3	5	25	5
---	---	----	---

4	3	12	4
---	---	----	---

包含 2

w=11

X

包含 1

w=4, v=40

ub=76

w=0, v=0

ub=100

不包含 1

w=0, v=0

ub=60

不包含 2

w=4, v=40

ub=70

包含 3

w=9, v=65

ub=69

不包含 3

w=4, v=40

ub=64

包含 4

w=12

X

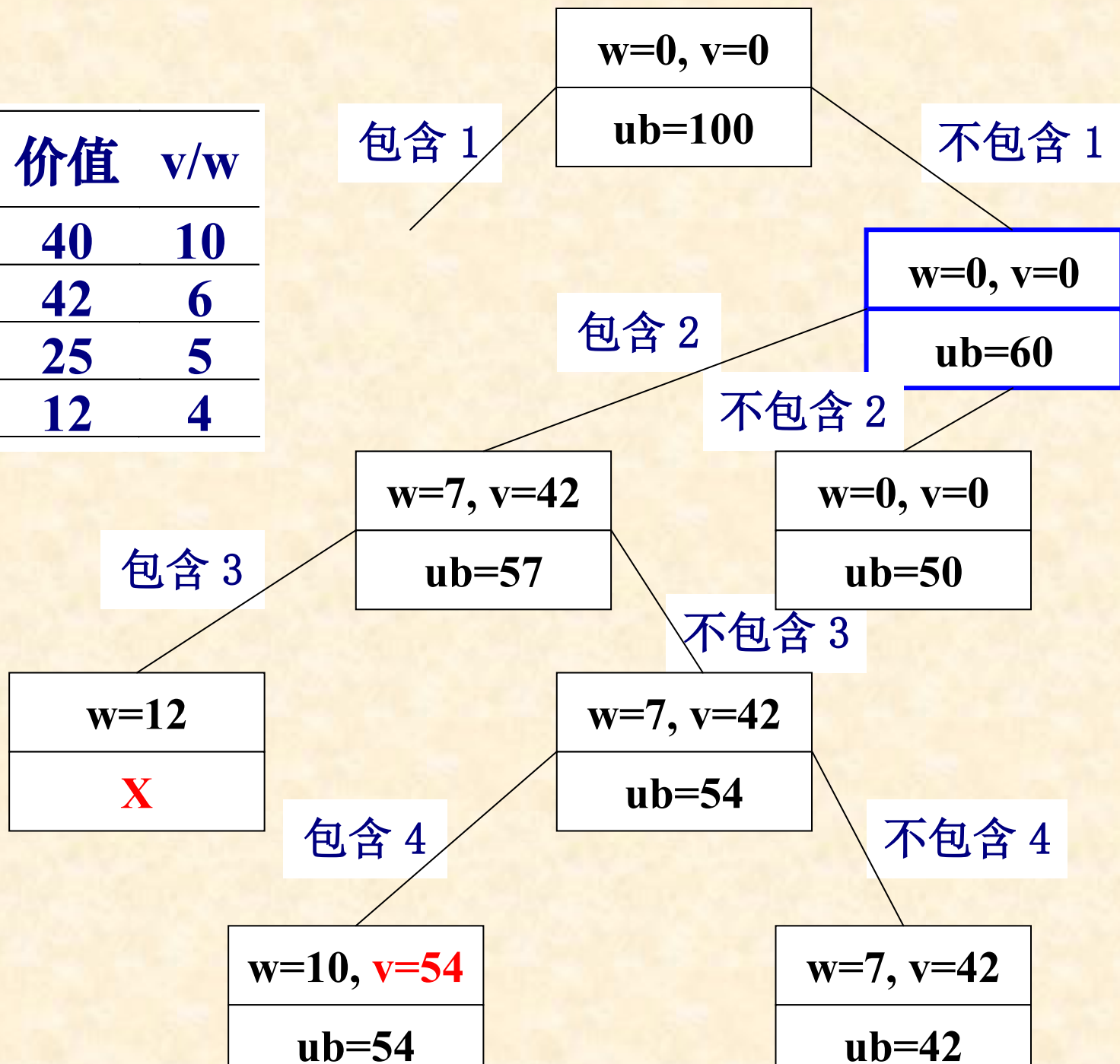
不包含 4

w=9, v=65

ub=65



物品	重量	价值	v/w
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4





物品	重量	价值	v/w
----	----	----	-----

1	4	40	10
---	---	----	----

2	7	42	6
---	---	----	---

3	5	25	5
---	---	----	---

4	3	12	4
---	---	----	---

包含 2

w=11

X

包含 1

w=4, v=40

ub=76

w=0, v=0

ub=100

不包含 1

w=0, v=0

ub=60

不包含 2

w=4, v=40

ub=70

包含 3

w=9, v=65

ub=69

不包含 3

w=4, v=40

ub=64

包含 4

w=12

X

不包含 4

w=9, v=65

ub=65



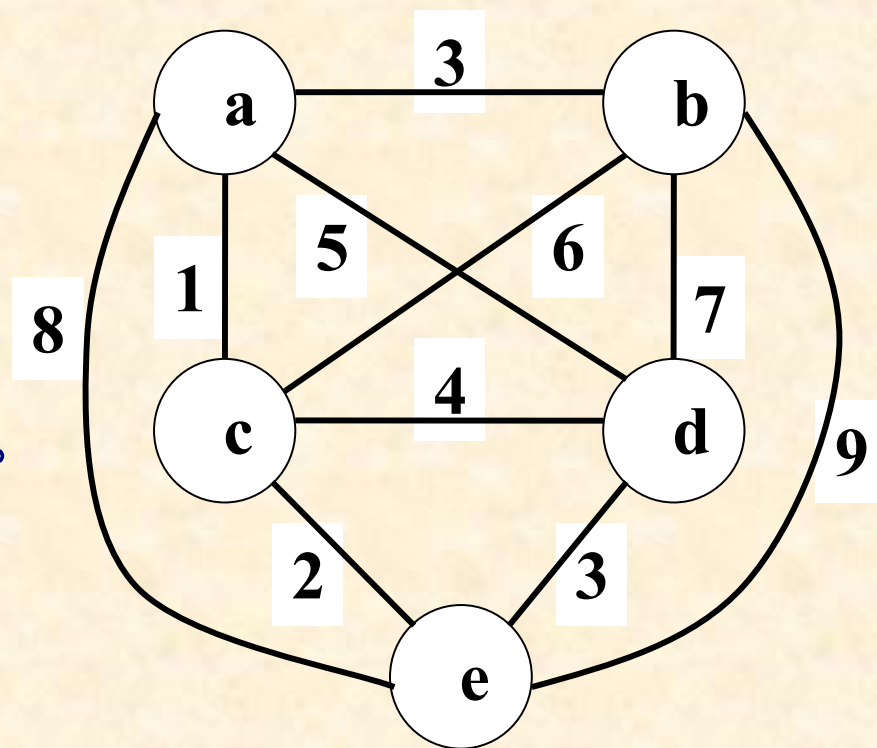
旅行商问题(TSP)

问题描述：一个商人要到 n 个城市去推销货物，从某城市出发，要求找到一条**闭合路径**， n 个城市都经历一次，最后回到出发点，且使得旅行的**花费最小**。

考虑 $n=5$ 的情形，如右图。

与背包问题（价值最大化）相反，
本问题（代价最小化）要估计下界。

简单下界：距离矩阵 D 的最小元素
乘以 n 即可。





一个信息量更大的下界：对每个城市 i ，计算 i 到最近的两个城市距离之和 s_i ，对 s_i 求和得到 s ，然后除以 2（或需取整），i.e.,

$$lb = \lceil s/2 \rceil \quad (\text{e.g., } n=3,4)$$

如右图，代价的下界

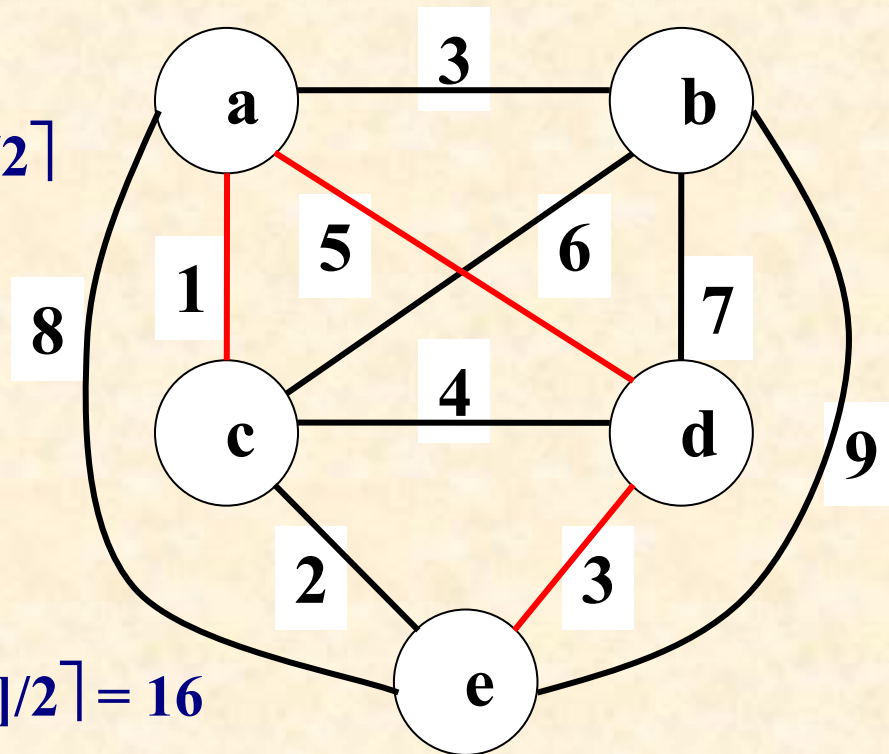
$$lb = \lceil [(1+3)+(3+6)+(1+2)+(3+4)+(2+3)]/2 \rceil = 14$$

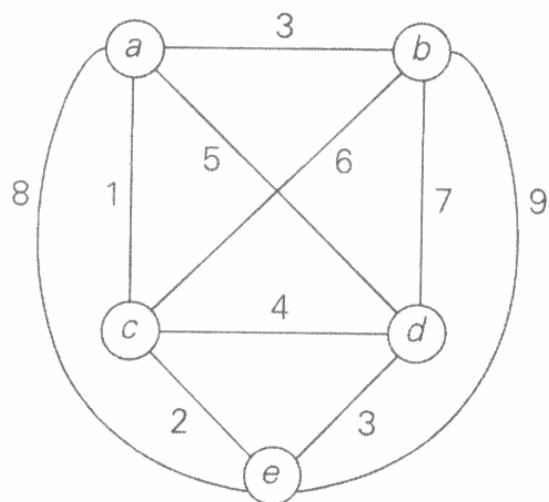
若必须包含某条边，如 (a,d) ，则把

a,d 两点分别邻接的两条最短边

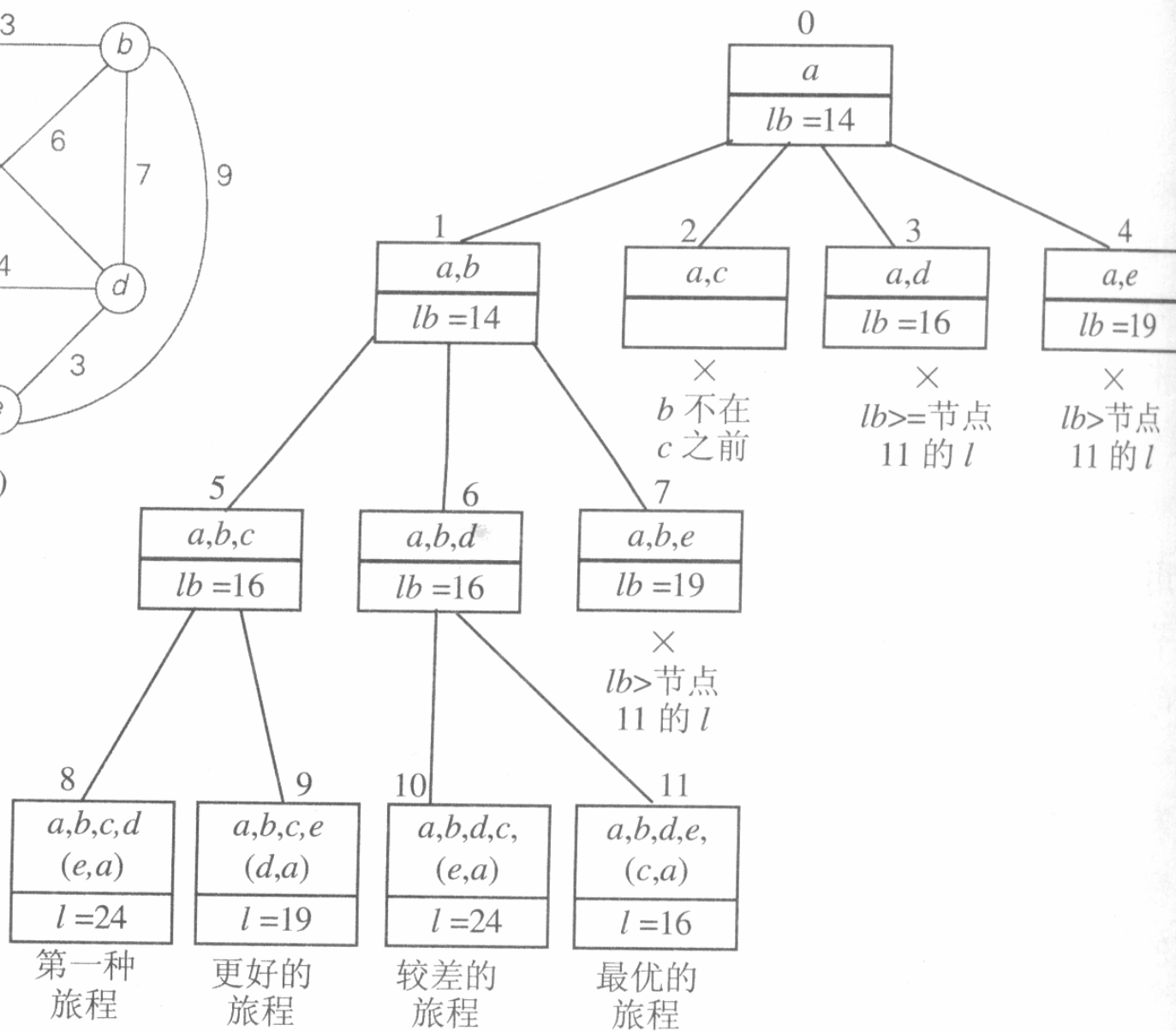
(a,c) , (c,d) 与 (a,d) , (d,a) 相加，即

$$lb = \lceil [(1+5)+(3+6)+(1+2)+(3+5)+(2+3)]/2 \rceil = 16$$





(a)





分枝限界法小结

- 分枝限界法的关键之处在于：**1)**如何选择节点的生成顺序；**2)**如何找到一个好的边界函数。
- 其中第**1**个方面是人工智能特别关心的，即生成状态空间树的不同策略。
- 另外，找到一个好的边界函数往往并不容易。一方面，我们希望这个函数比较容易计算，另一方面，又不能过于简单，否则无法完成其主要使命，即尽可能地修剪状态空间树的分枝。对于具体问题实例，需要进行大量试验，才能在这两个矛盾的需求之间达成折衷平衡。



作业

- 实验：给出高斯8皇后问题的所有解，要求使用递归和非递归算法，给出时间（单位：s）对比，并研究 $n=9, 10$ 的情况，若有解，请分别给出至少一个合法解。